FAST – NUCES

Object Oriented Programming Lab Manual

Course Teacher	Ms. Hina Iqbal
Lab Instructor	Mr. Ahmad Jawad Mustasim
Lab TA	Ms. Mania Shakeel
Section	BSE-2B
Semester	Spring 2026

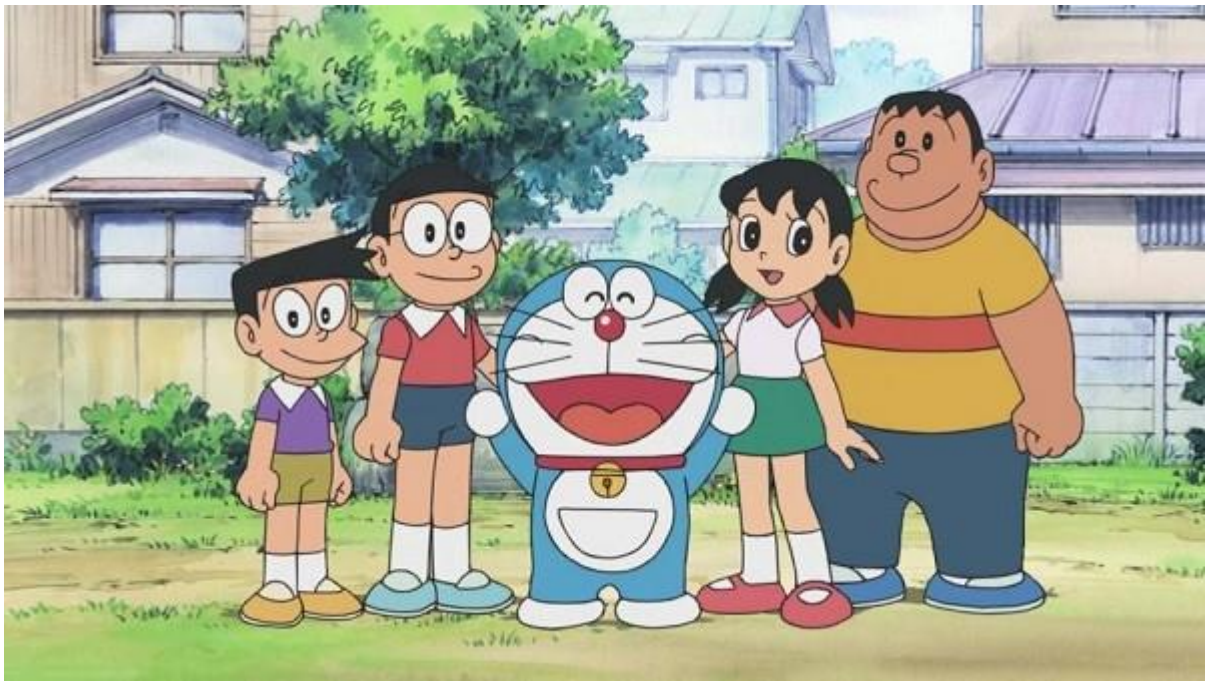
Department of Computer Science

FAST-NU, Lahore, Pakistan

Lab 10 – Association (Composition/Aggregation)

Objectives

- Apply core object-oriented programming concepts in C++ using a single class.
- Practice dynamic memory management with char*, and implement copy/move constructors for deep copying.
- Use constructors, destructors, inline functions, function overloading, static members, chain mutators, and the this pointer effectively.
- Introduction to Association(Composition/Aggregation) in OOP



Read carefully and attempt following task

Doraemon Gadget Management System

Scenario

In the futuristic world of Doraemon, gadgets are used by humans and robots.

In this world, there are **two types of humans** and **robots**, each interacting with gadgets differently:

Types of Humans

1. Normal Human (Borrowing – Aggregation)

- Can **borrow only ONE gadget** (not more than one)
- Does **NOT own** the gadget
- Gadget exists independently

If Human is destroyed → gadget still exists

2. VIP Human (Ownership – Composition)

Example: Nobita Nobi

- Can **own exactly ONE gadget** (not more than one)
- Gadget is part of VIPHuman

If VIPHuman is destroyed → gadget is also destroyed

Robots

Examples: Doraemon, Dorami

Robots can use gadgets in **two ways**:

Owned Gadgets (Composition)

- Robot can own **maximum 10 gadgets**
- Stored inside robot
- Automatically destroyed with robot

Shared Gadgets (Aggregation)

- Robot can reference **maximum 5 gadgets**
- Not owned by robot
- Not deleted by robot

Important Concept

If Doraemon is destroyed:

- All his **owned gadgets** are destroyed
- Shared gadgets still exist

Implement the following classes:

1. Gadget
2. Human (Borrowing – Aggregation)
3. VIPHuman (Ownership – Composition)
4. Robot (Both)

Also maintain a **static count of gadgets**

Class 1: Gadget

Attributes

int code;
char* name;
bool isActive;
static int gadgetCount;

Function Prototypes

Constructors

Gadget();
Gadget(int code, const char* name, bool isActive);
Gadget(const Gadget& other);
Gadget(Gadget&& other);

Destructor

~Gadget();

Setters (Chaining Required)

```
Gadget& setCode(int c);
Gadget& setName(const char* n);
Gadget& setActive(bool status);
```

Getters (Inline)

```
int getCode() const;
const char* getName() const;
bool getActive() const;
```

Display

```
void display() const;
```

Helper Functions

```
int strLength(const char* str);
void strCopy(char* dest, const char* src);
```

Static Function

```
static int getGadgetCount();
```

Example Output

Gadget ID: 101 | Name: Anywhere Door | Active: Yes

<u>Class 2: VIPHuman</u> (Ownership – Composition)

Attributes

```
char* name;
Gadget gadget;
```

Function Prototypes

```
VIPHuman();
VIPHuman(const char* n, int code, const char* name, bool isActive);

~VIPHuman();

VIPHuman& setName(const char* n);
VIPHuman& setGadget(int code, const char* name, bool isActive);

const char* getName() const;
Gadget* getGadget() const;

void display() const;
```

Example Output

```
VIP Human: Nobita
Borrowed Gadget:
Gadget ID: 101 | Name: Bamboo Copter | Active: Yes
```

<u>Class 3: Human</u> (Borrowed – Aggregation)

Attributes

```
char* name;
Gadget* gadget;
```

Function Prototypes

```
Human();
Human(const char* n, const Gadget& g);

~Human();

Human& setName(const char* n);
Human& setGadget(const Gadget& g);

const char* getName() const;
Gadget& getGadget();

void display() const;
```

Example Output

```
Human: Shizuka
Owned Gadget:
Gadget ID: 102 | Name: Anywhere Door | Active: Yes
```

Class 4: Robot

Attributes

```
char* name;

// Composition
Gadget ownedGadgets[10];
int ownedCount;

// Aggregation
Gadget* sharedGadgets[5];
int sharedCount;
```

Function Prototypes

```
Robot();
Robot(const char* n);

~Robot();

Robot& setName(const char* n);

Robot& addOwnedGadget(int code, const char* name, bool isActive);
Robot& addSharedGadget(Gadget* g);

const char* getName() const;

void display() const;
```

Example Output

Robot Name: Doraemon

Owned Gadgets:
- Gadget ID: 301 | Name: Translator Tool | Active: Yes
- Gadget ID: 302 | Name: Time Freezer | Active: No

Shared Gadgets:
- Gadget ID: 201 | Name: Bamboo Copter | Active: Yes

Testing in Main Function

Step 1: Input Number of Gadgets

How many gadgets would you like to create?

Step 2: Create Dynamic Array

Gadget* gadgets = new Gadget[n];

Step 3: Input Gadget Data

For each gadget:

- code
- name
- active status

Use setters with chaining

Step 4: Display All Gadgets

--- All Gadgets ---
Gadget ID: ...

Step 5: Create Humans

- Create **at least 2 Humans** (Shizuka, Gian or Suneo)
- Assign borrowed gadgets using pointers

Step 6: Create One VIPHuman (Nobita)

- Assign one gadget

Step 7: Create Robots

- Create at least **1 robot** (Doraemon and Dorami)
- Add owned and shared gadgets

Step 8: Display Everything

--- Humans ---
...

--- VIP Human ---
...

--- Robots ---
...

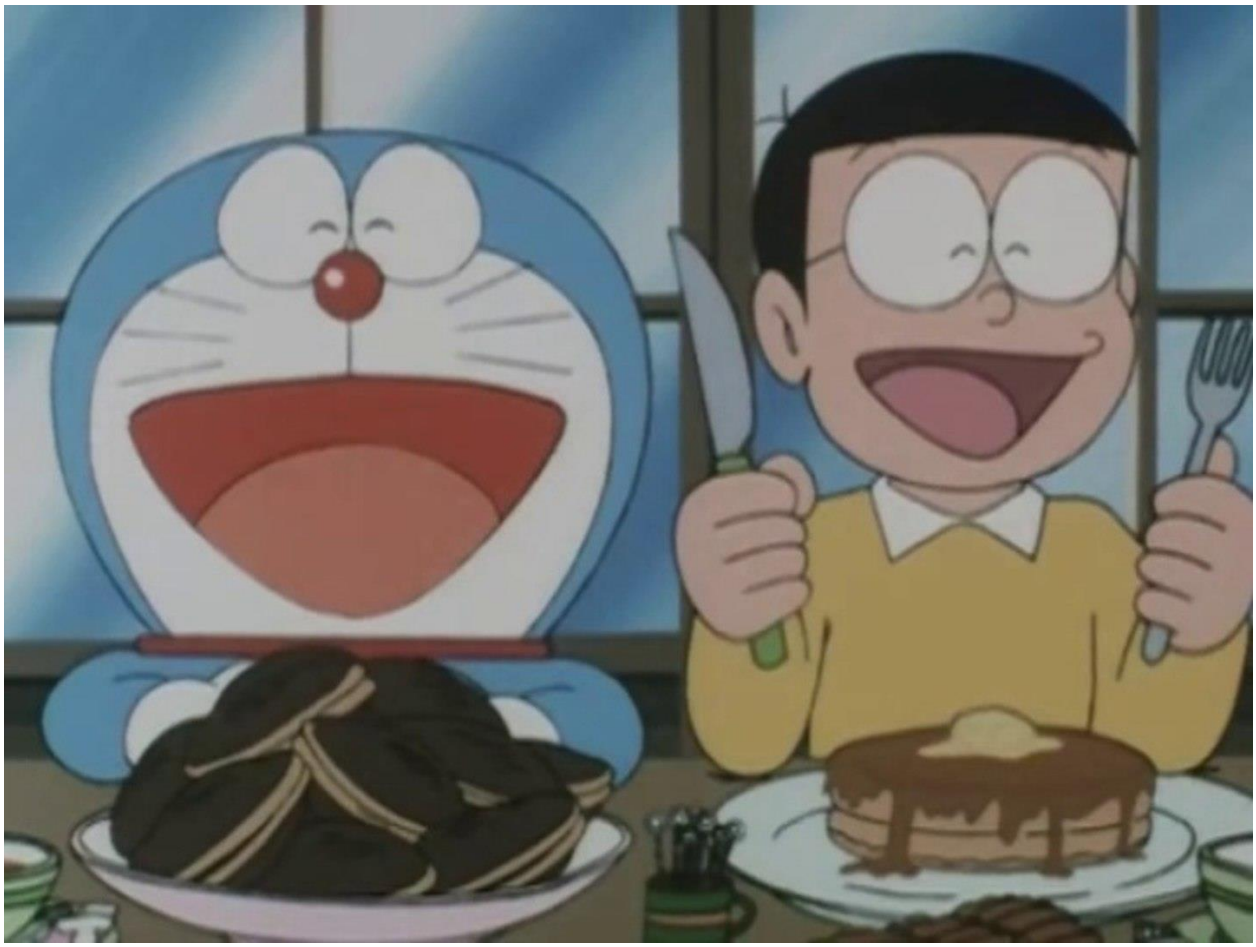
Step 9: Display Total Gadgets

Total Gadgets Created: X

Step 10: Cleanup

delete[] gadgets;

----------Happy Coding! ☺----------

----------**Submission Instructions**----------

- Submit your tasks on Google Classroom within given deadline
- Implement all tasks in same .cpp file
- Also submit screenshots of each task for example “LAB01-TASK01.png”, “LAB01-TASK02.png” and so on
- “.cpp” is the actual file where all your code is saved.
- Always submit .cpp files when asked for submission